P2713R0
Escaping improvements in std::format

Published Proposal, 2022-11-09

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Proposal

This paper provides wording for the resolution of national body comments [US38-098] and [FR005-134] per direction voted in SG16 Unicode and LEWG. The direction is summarized in [US38-098]:

> The first poll confirms the intent that an escaped string be usable as a string literal (e.g., that it can be copied and pasted into a C++ program) such that, when evaluated as a string literal, the input used to produce the escaped string is reproduced. No changes are required to satisfy this poll.
>
> The second poll clarifies that it is intended that the escaped string be readable by humans. The context for this poll was concern about producing visually ambiguous output. The SG16 conclusion is that escaped strings are not intended to produce visually unambiguous results; it is ok for the escaped string to contain unescaped characters that might be confused with other characters (e.g., characters considered "confusables" by Unicode). No changes are required to satisfy this poll.
>
> The third poll clarifies that it is intended that separator and non-printable characters continue to be escaped. No changes are required to satisfy this poll.
>
> The last poll indicates a change in direction relative to the current wording. SG16 desires that combining characters (those with the Unicode property `Grapheme_Extend=Yes`) shall be escaped if they are not preceded by a non-escaped lead character (or another combining character that is preceded by a lead character). Satisfying this poll will require normative changes to [format.string.escaped]p2.

SG16 poll results for [US38-098]:

Poll 2.1: [US 38-098] SG16 agrees that the formatted code units in the escaped string are intended to be usable as a string literal that reproduces the input.
Attendees: 8
No objection to unanimous consent.

Poll 2.2: [US 38-098] SG16 agrees that the escaped string is intended to be readable for its textual content in any Unicode script.
Attendees: 8
No objection to unanimous consent.

Poll 2.3: [US 38-098] SG16 agrees that separators and non-printable characters ([format.string.escaped]p(2.2.1.2)) shall be escaped in the escaped string.
Attendees: 8
No objection to unanimous consent.

Poll 2.4: [US 38-098] SG16 agrees that combining code points shall not be escaped unless there is no leading code point or the previous character was escaped.
Attendees: 8
No objection to unanimous consent.

SG16 poll results for [FR005-134]:

Poll 1: [FR 005-134]: SG16 recommends accepting the comment in the direction presented in the first bullet of the proposed change and as recommended in the polls for US 38-098.
Attendees: 8
Unanimous consent

LEWG poll results:

POLL: We agree with the direction of the proposed SG16 recommendation for US 38-098 & FR005-134.

SF	F	N	A	SA
9	7	0	0	0

Outcome: Unanimous consent

§ 2. Wording

In [format.string.escaped]:

1 A character or string can be formatted as *escaped* to make it more suitable for debugging or for logging.

2 The escaped string *E* representation of a string *S* is constructed by encoding a sequence of characters as follows. The associated character encoding *CE* for `charT` (Table 13) is used to both interpret *S* and construct *E*.

- U+0022 QUOTATION MARK (") is appended to *E*.
- For each code unit sequence *X* in *S* that either encodes a single character, is a shift sequence, or is a sequence of ill-formed code units, processing is in order as follows:
  - If *X* encodes a single character *C*, then:
    - If *C* is one of the characters in Table 76, then the two characters shown as the corresponding escape sequence are appended to *E*.
    - Otherwise, if *C* is not U+0020 SPACE and
      - ~~*CE* is a Unicode encoding and *C* corresponds to either a UCS scalar value whose Unicode property `General_Category` has a value in the groups `Separator` (Z)~~

~~or `Other` (C) or to a UCS scalar value which has the Unicode property `Grapheme_Extend=Yes`, as described by table 12 of UAX #44, or~~

- *CE* is a Unicode encoding and *C* corresponds to a UCS scalar value whose Unicode property `General_Category` has a value in the groups `Separator` (Z) or `Other` (C), as described by table 12 of UAX #44, or
- *CE* is a Unicode encoding and *C* corresponds to a UCS scalar value which has the Unicode property `Grapheme_Extend=Yes` and is not immediately preceded in *S* by a UCS scalar value appended to *E* or
- *CE* is not a Unicode encoding and *C* is one of an implementation-defined set of separator or non-printable characters

then the sequence `\u{hex-digit-sequence}` is appended to *E*, where *hex-digit-sequence* is the shortest hexadecimal representation of *C* using lower-case hexadecimal digits.

- Otherwise, *C* is appended to *E*.

- Otherwise, if *X* is a shift sequence, the effect on *E* and further decoding of *S* is unspecified.

  Recommended practice: A shift sequence should be represented in *E* such that the original code unit sequence of *S* can be reconstructed.

- Otherwise (*X* is a sequence of ill-formed code units), each code unit *U* is appended to *E* in order as the sequence `\x{hex-digit-sequence}`, where *hex-digit-sequence* is the shortest hexadecimal representation of *U* using lower-case hexadecimal digits.

- Finally, U+0022 QUOTATION MARK (") is appended to *E*.

...

[*Example 1*:

```
string s0 = format("[{}]", "h\tllo");               // s0 has value: [h
string s1 = format("[{:?}]", "h\tllo");             // s1 has value: ["h\tl
string s3 = format("[{:?}, {:?}]", '\'', '"');      // s3 has value: ['\'',

// The following examples assume use of the UTF-8 encoding
string s4 = format("[{:?}]", string("\0 \n \t \x02 \x1b", 9));
                                                    // s4 has value: ["\u{0
string s5 = format("[{:?}]", "\xc3\x28");           // invalid UTF-8, s5 ha
```

```
string s6 = format("[{:?}]", "\u0301");             // s6 has value: ["\u{3
string s7 = format("[{:?}]", "\\\u0301");           // s7 has value: ["\\\u
string s8 = format("[{:?}]", "e\u0301\u0323");      // s8 has value: ["ẹ́"]
```

— *end example*]

3. Acknowledgements

Thanks to Tom Honermann for nicely summarizing the resolution of NB comments in [US38-098] which is quoted in this paper.

References

Informative References

[FR005-134]

FR 005-134 22.14.6.4 [format.string.escaped] Aggressive escaping. URL: https://github.com/cplusplus/nbballot/issues/408

[US38-098]

US 38-098 22.14.6.4p1 [format.string.escaped] Escaping for debugging and logging. URL: https://github.com/cplusplus/nbballot/issues/515